

The 'var' Keyword (Java 10+)

Type Inference: Letting Java do the work for you

Introduced in **2018 (Java 10)**, `var` allows the compiler to "guess" the data type automatically based on what value you assign to it. This is called **Type Inference**.

1. Automatic Detection

Instead of typing `int`, `String`, or `double`, you can just use `var`.

```
var age = 25; // Compiler sees 25, so age is an int
var price = 19.99; // price becomes a double
var name = "Codehive"; // name becomes a String
var isReady = true; // isReady becomes a boolean
```

2. The Literal Rules

While `var` makes code shorter, it has two strict rules you must follow:

Rule 1: Immediate Assignment

You **must** give it a value immediately so Java knows the type.

```
var x = 10; ✓
```

Rule 2: No Late Assignment

You cannot declare it empty. Java won't know what it's supposed to be.

```
var x; ✗ (Error)
```

3. Type Lock-In

Even though `var` looks dynamic (like JavaScript or Python), Java is still **Strongly Typed**. Once the variable's type is inferred, it is locked forever.

```
var score = 100; // score is now an 'int'
score = 200; // Works fine
score = "Win"; // ERROR! You cannot change an int to a String
```

4. Why use var?

The primary reason for `var` is to avoid "Boilerplate" code—the repetitive typing of long class names.

Old Way:

```
ArrayList<String> list = new ArrayList<String>();
```

Modern Way with var:

```
var list = new ArrayList<String>();
```

Summary Checklist

- Use `var` to make declarations shorter and cleaner.
- It is available in **Java 10** or later versions.
- You **must** initialize the variable (`var x = 5;`).
- The type is fixed once assigned and **cannot change**.
- Use it mostly for complex types to keep code readable.